

KubeEdgeInfer

A Closed-Loop, eBPF-Driven Kubernetes Framework for Heterogeneous
Distributed LLM Inference on Consumer Edge Clusters

Your Name

Department of _____

Guide: Sir's Name

Date

Introduction

- Large Language Models (LLMs) are often too large to fit on a single consumer GPU.
- **Pipeline parallelism** solves this by splitting model layers across several machines, forming an inference assembly line.
- Existing frameworks are built either for stable datacenter interconnects, or they split layers **statically** and never revisit that decision.
- Consumer and edge hardware is inherently **heterogeneous**: different GPU generations, WiFi rather than Ethernet, and thermal variance under sustained load.

Core Idea

Continuously observe real hardware conditions at the kernel level, and let the model's layer partition adapt to them instead of staying fixed.

Literature Review

Work	Year	Contribution	Gap this work addresses
EdgeShard	2024	DP device selection and layer partition	Profiled offline; plan never revisited
Galaxy	2024	Tensor/sequence parallelism with overlap	One-shot planning, no adaptation
Helix	2025	Max-flow MILP over mixed GPUs	Solver too heavy for a live loop
TPI-LLM	2024	Memory scheduling for 70B on small RAM	Optimises memory, not degradation
prima.cpp	2025	CPU/GPU co-scheduling on home clusters	Schedule fixed at load time
Head-level split	2025	Attention-head granularity, with migration	Triggered by memory pressure only
Parallax	2025	Serving over volunteer machines	No kernel-level measurement
Adaptive split (MBRL)	2025	Learns the split point as the channel varies	Needs training; no optimality proof
Distributed LLM survey	2025	Surveys the field	Names adaptation as an open problem
sched_ext (eBPF)	2025	Linux schedulers loaded as eBPF programs	Single host; no model pipeline

Common limitation

Every one of these measures the hardware before the run, and never measures again.

Problem Statement

Static partitioning ignores real-time hardware variability.

Thermal Throttling

GPUs slow down mid-inference as they heat up under sustained load.

Network Latency

WiFi and edge links have variable latency — no InfiniBand or NVLink at home.

Node Failure

A laptop closes its lid or drops off the network, and its layers go with it.

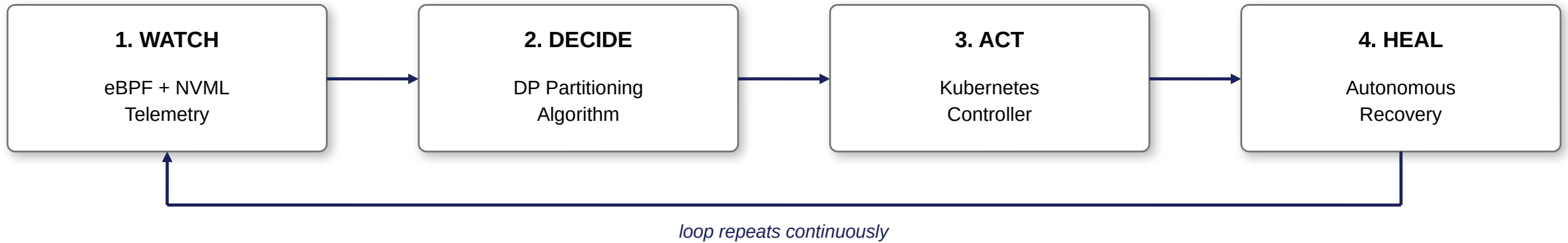
Consequence: Pipeline Bubble Time

Fast machines sit idle waiting on slower ones. A one-time, fixed split cannot correct itself as conditions change.

Objectives and Contributions

- 1 Capture system telemetry — network transfer latency, GPU thermal state, throttling — using **eBPF** and **NVML**, with no change to the model code.
- 2 Design a **dynamic partitioning algorithm** that computes the optimal contiguous layer split from live telemetry, and prove it optimal for each snapshot.
- 3 Integrate telemetry and partitioning decisions directly into a **Kubernetes** control loop, applying them without restarting pods.
- 4 Enable **autonomous healing**: automatic re-partitioning on node failure or sustained thermal throttling.
- 5 Stabilise the loop with **hysteresis** — a 15% improvement threshold and a 30 s cooldown — so noisy telemetry cannot cause thrashing.
- 6 Evaluate bubble time, throughput and time-to-first-token empirically against static partitioning and an offline-profiling baseline.

Architecture — How It Works



- **Watch:** kernel-level flow latency, GPU temperature, memory, throttle state.
- **Decide:** compute the optimal contiguous split from live metrics.
- **Act:** re-assign layers, reroute traffic, apply the new split with no restart.
- **Heal:** detect node failure within 3 s, re-partition across the survivors.

Results

Injected condition	KubeEdgeInfer	Profile-once	Static split
No fault	42 ms	42 ms	42 ms
Thermal throttle (0.4x)	52 ms	102 ms	102 ms
Network delay (80 ms)	62 ms	122 ms	122 ms
Thermal + network	62 ms	182 ms	182 ms
Node failure	62 ms	62 ms	pipeline stops

Bottleneck-stage cost per token; lower is better. 12 layers over 3 machines.

Headline

49% lower bottleneck cost under thermal throttling and under network degradation; 66% lower when both occur together. Under node failure the static split is undefined over the surviving set, so the pipeline stops — KubeEdgeInfer re-partitions and continues.

Reproducible with `go run ./bench/dpsim`; end-to-end throughput comes from `bench/run.py`, which needs a live cluster.

Feature	EdgeShard	Galaxy	Helix	prima.cpp	Ours
Heterogeneous edge support	✓	✓	✗	✓	✓
Kernel-level telemetry (eBPF)	✗	✗	✗	✗	✓
Dynamic re-partitioning	✗	✗	✗	✗	✓
Thermal awareness	✗	✗	✗	✗	✓
Autonomous healing	✗	✗	✓	✗	✓
Declarative K8s API	✗	✗	✗	✗	✓

One-line novelty statement

We turn a static, one-time model-splitting decision into a continuous, self-correcting one — driven by real kernel-level measurements instead of assumptions.

Summary

- First system to close the loop: eBPF/NVML telemetry → Kubernetes scheduling → self-healing, for edge LLM inference.
- The partitioner is **provably optimal** for a fixed telemetry snapshot — it reduces to the classical linear partition problem, and is unit-tested against brute force on 200 random instances.
- Adaptive behaviour is validated **empirically** through a staged ablation under simulated network, thermal and failure conditions.
- Distributed GPT-2 output is **token-identical** to single-process inference, so adaptivity costs no correctness.
- Fully software-based and reproducible — no proprietary hardware lab required.

Thank you — Questions?

References

- [1] M. Zhang, J. Cao, X. Shen, Z. Cui. EdgeShard: Efficient LLM Inference via Collaborative Edge Computing. IEEE Internet of Things Journal, 2024. arXiv:2405.14371.
- [2] Galaxy: A Resource-Efficient Collaborative Edge AI System for In-situ Transformer Inference. IEEE INFOCOM, 2024. arXiv:2405.17245.
- [3] Helix: Serving Large Language Models over Heterogeneous GPUs and Network via Max-Flow. ACM ASPLOS, 2025. arXiv:2406.01566.
- [4] TPI-LLM: Serving 70B-scale LLMs Efficiently on Low-resource Edge Devices. 2024. arXiv:2410.00531.
- [5] Prima.cpp: Fast 30-70B LLM Inference on Heterogeneous and Low-Resource Home Clusters. 2025. arXiv:2504.08791.
- [6] Large Language Model Partitioning for Low-Latency Inference at the Edge. 2025. arXiv:2505.02533.
- [7] Parallax: Efficient LLM Inference Service over Decentralized Environment. 2025. arXiv:2509.26182.
- [8] Adaptive layer splitting for wireless large language model inference in edge computing. FITEE, Springer, 2025. doi:10.1631/FITEE.2400468.
- [9] Distributed LLMs and Multimodal Large Language Models: A Survey on Advances, Challenges, and Future Directions. 2025. arXiv:2503.16585.
- [10] Towards Agentic OS: An LLM Agent Framework for Linux Schedulers. 2025. arXiv:2509.01245.

Tools: cilium/ebpf (CO-RE loader), Kubernetes controller-runtime, NVIDIA NVML, k3s, Hugging Face Transformers.